hope that the program will do what it should. The broken tracking application showed you that hope isn't an effective long-term testing strategy. How do you write a test for the requirement?

To learn how to test effectively, you need to learn about assertions, how to write them for the smallest parts (units) of your program, how to repeatedly write them as you develop your program, and how to get reliable feedback to tell you whether your assertions hold or not. The first step is understanding assertions.

10.2.1. Assertions

For a moment, forget about how you'll implement the tracking application. Imagine that it's already written and that as part of your work you have a `totalVisits` variable that tells you how many visits have been recorded. The test would follow along these lines:

```
initial = totalVisits
# Somehow visit the homepage
# Assert that the totalVisits has increased by one
```

There are two parts of this that you don't yet know how to write in CoffeeScript: write a program that visits the homepage (like a real user), and assert that the number of visits has increased afterward. You'll learn assertions first.

An *assertion* is a statement about the value of an expression. It either passes or fails. CoffeeScript doesn't have assertions built-in, but Node.js has a built-in assert module that you can use. To use the `assert` module you `require` it:

```
assert = require 'assert'
# { [Function: ok] ...
```

Then you invoke one of the assert methods, such as `assert.ok`, which tests whether an expression is truthy. If the assertion succeeds, it evaluates to `undefined`: